
title: "Memory Performance in AI Image and Text Generation: Comparing 4-bit and 8-bit Models on Apple M2"

author: "Saurav Riz"

date: "April 28, 2026"

Memory Performance in AI Image and Text Generation

Comparing 4-bit and 8-bit Models on Apple M2

Author: Saurav Riz

Date: April 28, 2026

Abstract

This report explores how the Apple M2 chip handles memory during AI tasks involving both images and text. We compare two ways of compressing AI models: 4-bit and 8-bit compression. Using Apple's built-in power tracking tool, we collected over a million data points across three tests: text-only, single image with text, and a five-frame video sequence. Our goals were to find out when the M2's memory system reaches its limit, how the graphics processor (GPU) transitions between processing images and generating text, and whether using the more compressed 4-bit model slows down the GPU because it has to decompress the data while running.

Our main findings are: (1) The memory limit is hit hardest when the AI is first reading the prompt, not when it is generating the response. (2) The GPU usually runs at its lowest speed, meaning it is waiting for data from memory rather than doing complex calculations. (3) The time taken to decompress the 4-bit model is noticeable but completely hidden by the time spent waiting for memory. Ultimately, using the 4-bit compression is the best choice for the M2 chip because it halves the memory traffic without causing a noticeable slowdown in processing.

1. Introduction

1.1 Background

Running large AI models on everyday computers has become much easier. A big reason for this is "compression," a technique that shrinks the size of the AI's internal numbers from larger sizes to smaller 8-bit or 4-bit sizes. On traditional computers with separate graphics cards, this creates a trade-off: smaller sizes mean less data to move, but the graphics card has to spend extra effort unpacking the smaller data before using it.

Apple's M2 chip works differently. It uses "Unified Memory," meaning the main processor (CPU), the graphics processor (GPU), and the AI processor (Neural Engine) all share the same memory pool. Because they all use the same pathway to get data, memory traffic can easily become a bottleneck.

1.2 The Problem

When running these AI models on an Apple M2 chip, does choosing 4-bit over 8-bit compression change how the system handles memory, and is there a noticeable performance penalty for having to unpack the 4-bit data?

1.3 Goals

We have three main goals:

- **Goal 1 (Memory Limits):** Find out when the memory system reaches its maximum capacity during different AI tasks.
 - **Goal 2 (Processor Shifts):** Observe how the workload shifts between the CPU and GPU when moving from analyzing an image to generating text.
 - **Goal 3 (Unpacking Penalty):** Measure if there is a real slowdown caused by unpacking 4-bit data on the fly.
-

2. Background Information

2.1 AI on Everyday Computers

When running AI models one request at a time, the main limitation is usually how fast data can be moved from memory to the processor, rather than how fast the processor can calculate. Because the AI has to constantly fetch its massive list of parameters, it spends most of its time waiting for data.

2.2 Compression Methods

We compared two compression levels: 4-bit (which is very efficient and commonly used) and 8-bit (which is less compressed but closer to the original model). The 8-bit version requires moving nearly twice as much data as the 4-bit version. On the M2 chip, moving more data means using more power and potentially slowing down the system.

2.3 Measuring Hardware Performance

To measure this, we used Apple's built-in tools to track how much power each part of the chip was using and how fast the GPU was running. If the GPU is running at its minimum speed but is still active, it usually means it is waiting for data from memory.

3. How We Tested

3.1 The Setup

We used an Apple Mac Mini with the M2 chip and 8 GB of unified memory. We tested a specific AI model that can understand both images and text.

3.2 The Tests

We ran three different tests:

1. **Text-Only:** Asking the AI to write a long text response. This tests the system without any image processing.
2. **Single Image:** Asking the AI to describe one image in detail. This lets us see the shift from image processing to text generation.

3. **Video Sequence:** Asking the AI to describe five frames of a video in a row. This creates a repeating pattern of image and text processing.

Table 1: Overview of AI Tests and Peak Power Usage

Test Scenario	Total Data Points	4-bit Peak Power (mW)	8-bit Peak Power (mW)	Power Difference
Text-Only	19,167	20,554	23,637	+15%
Single Image	96,051	16,131	18,550	+15%
Video (5 Frames)	441,015	19,781	22,748	+15%

3.3 Data Collection

We collected hardware data twice a second, tracking power usage, temperature, and how fast the processors were running. We then categorized the data into three phases: "image processing," "text generation," and "idle time."

4. What We Found

4.1 Goal 1: Memory Limits

As shown in **Table 1**, the most intense pressure on the memory system happens right at the beginning, when the AI is reading the user's prompt. The text-only test actually consumed the most power at its peak because reading a long text prompt forces the CPU to work incredibly hard and use a lot of memory all at once. The 8-bit model used about 15% more peak power than the 4-bit model, which makes sense because it has to load larger files.

4.2 Goal 2: Processor Shifts

There is a clear pattern when the AI switches from looking at an image to writing text. During image processing, the GPU does a lot of the work. But as soon as it starts generating text, the GPU's activity drops significantly, and the CPU takes over almost entirely. The AI Neural Engine also turns on specifically during image processing but shuts off when generating text.

Table 2: Processor Workload Shift (4-bit Model)

Phase	Graphics Processor (GPU) Activity	Main Processor (CPU) Power
Analyzing Image	High (~31% active time)	Normal (~1,500 mW)
Generating Text	Low (~6% active time)	Very High (~3,148 mW)

4.3 Goal 3: Unpacking Penalty

We wanted to see if the 4-bit model slowed the GPU down because of the extra work required to unpack the compressed data. We found that the GPU spends almost all of its active time running at its absolute minimum speed (444 MHz) for both models. This means the GPU is constantly waiting for memory, rather than actively calculating.

Table 3: GPU Waiting Time (% of time spent at lowest speed)

Test Phase	4-bit Model Wait Time	8-bit Model Wait Time
Reading Prompt (Text-Only)	80.5%	77.8%
Generating Text (Text-Only)	91.4%	93.4%

Analyzing Image	74.7%	73.5%
Generating Text (Image Test)	86.2%	85.5%

While there were tiny differences in how long the GPU waited between the 4-bit and 8-bit models (as seen in **Table 3**), the time spent unpacking the 4-bit data was completely hidden by the time spent waiting for memory to arrive. The GPU never reached high speeds, meaning it was never maxed out by complex math.

5. What This Means

5.1 Reading the Prompt is the Hardest Part

It is surprising that simply reading a text prompt puts more stress on the system than analyzing a complex image or video. This is because processing the prompt is handled by the CPU in one massive burst, whereas video processing is spread out over time.

5.2 The 4-bit Model is the Clear Winner

Using the 4-bit compression is the best choice for the M2 chip. It cuts the amount of data moving through the system nearly in half, which saves power and reduces memory traffic. The theoretical downside—that the computer has to work harder to unpack the data—does not matter in reality because the processor is already stuck waiting for memory anyway.

Table 4: Final Verdict: 4-bit vs. 8-bit Compression

Feature	4-bit Compression	8-bit Compression	Best Choice
Data Size (Bytes per weight)	~4.5	~8.5	4-bit (Half the size)
Peak Power Usage	Baseline	15% Higher	4-bit
Wait Time for Memory	Near-identical	Near-identical	Tie
Quality of AI Response	Slightly lower	Near perfect	8-bit

6. Conclusion

Our tests show that when running AI models on the Apple M2 chip, the system is always limited by how fast it can move data, not by how fast it can calculate.

The memory system is pushed to its limits when reading the initial prompt, not when generating the answer. The workload clearly shifts from the GPU (for images) to the CPU (for text). Most importantly, while the 4-bit model requires extra effort to unpack, this effort is completely masked by the time the system spends waiting for memory.

Therefore, 4-bit compression is the superior option for these devices. It significantly reduces memory traffic and power usage without causing any noticeable slowdown.

7. Recommendations

- **For Users:** Always use 4-bit compression when running AI models on everyday Apple computers. It is much more efficient than 8-bit and won't slow you down.
- **For Testing:** If you want to see how much an AI model strains a computer's memory, test it with very long text prompts rather than just having it generate long answers.

- **For the Future:** It would be helpful to test if these findings hold true when the computer is trying to handle multiple AI requests at the exact same time.